

Semana 2 Búsqueda Informada y ambientes complejos

ISIS-1611: Inteligencia Artificial

Carla González

9 de junio de 2026

1. Búsqueda Informada Avanzada

1.1. Búsqueda con Memoria Acotada

1.1.1. Iterative Deepening A* (IDA*)

Versión de A* que usa IDDFS con el valor de corte $f(n) = g(n) + h(n)$.

- Espacio: $\mathcal{O}(bd)$
- Completo y óptimo (con h admisible)

1.1.2. Recursive Best-First Search (RBFS)

Mantiene el mejor valor alternativo f en cada nivel de recursión.

- Espacio: $\mathcal{O}(bd)$
- Puede reexpandir nodos (regenera sub-árboles olvidados)

1.1.3. SMA* (Simplified Memory-Bounded A*)

Usa toda la memoria disponible; descarta el nodo con mayor f cuando la memoria está llena.

1.2. Heurísticas: Dominancia y Creación

Definición 1.1 (Dominancia). h_2 domina a h_1 si $h_2(n) \geq h_1(n)$ para todo n y ambas son admisibles. Una heurística dominante expande menos nodos.

Formas de construir heurísticas admisibles:

1. **Relajación del problema:** eliminar restricciones del problema original.
2. **Bases de datos de patrones:** precomputar costos exactos de subproblemas.
3. **Máximo de heurísticas:** $h(n) = \max(h_1(n), h_2(n))$

2. Búsqueda en Ambientes Complejos

2.1. Búsqueda Local

A diferencia de la búsqueda sistemática, la búsqueda local opera sobre un **estado actual** sin mantener el camino recorrido. Útil cuando el objetivo es el estado mismo (no el camino).

2.2. Hill Climbing (Ascenso de Colina)

Algorithm 1 Hill-Climbing

```

1: current ← estado inicial
2: loop
3:   neighbor ← mejor sucesor de current
4:   if  $f(\text{neighbor}) \leq f(\text{current})$  then return current
5:   end if
6:   current ← neighbor
7: end loop

```

Problemas:

- Máximos locales (no globales)
- Mesetas (*plateaus*) y crestas

Variantes: Hill climbing estocástico, reinicio aleatorio, *simulated annealing*.

2.3. Simulated Annealing

Permite movimientos malos con probabilidad $e^{\Delta E/T}$ donde T decrece con el tiempo. Garantiza convergencia al óptimo global si T baja suficientemente despacio.

2.4. Algoritmos Genéticos

Inspirados en la evolución biológica. Operan sobre una **población** de individuos (estados codificados como cadenas).

Operadores:

1. **Selección:** individuos con mayor aptitud (*fitness*) tienen más probabilidad de reproducirse.
2. **Cruzamiento** (*crossover*): combina dos individuos para producir descendencia.
3. **Mutación:** cambia aleatoriamente genes con baja probabilidad.

3. Ejercicios de Búsqueda

Nota

Esta clase es de ejercicios y repaso para el **Examen 1**. Ver la *Guía de Ejercicios: Búsqueda Desinformada e Informada*.

3.1. Guía de Ejercicios — Resumen de Bloques

3.1.1. Bloque 1: Agentes Inteligentes

Caracterizar escenarios con PEAS, identificar tipo de agente y entorno.

3.1.2. Bloque 2: Formulación de Problemas

Definir estado inicial, modelo de transición, costo y test de meta para problemas concretos.

3.1.3. Bloque 3: Búsqueda No Informada

Aplicar BFS, DFS y UCS sobre grafos; analizar completitud y optimalidad.

3.1.4. Bloque 4: Búsqueda Informada y Heurísticas

Aplicar Greedy, A*, RBFS, SMA*; verificar admisibilidad y consistencia de heurísticas.

Ejercicio 3.1. Dado el grafo del Bloque 3 (Guía), aplica BFS, DFS y UCS. Indica el orden de expansión y el camino solución encontrado.

Solución:

4. Heurísticas

Es una función que estima que tan cerca está un estado a su meta, guía para el camino con mayor promesa. Las heurísticas es que no sobrestimen los costos. Si dos estados tienen una evaluación heurística similar $\rightarrow$ preferiblemente examinar estado más cercano a la raíz en grafo de estados.

Definición 4.1. Una **heurística** es una función que estima que tan cerca está un estado n de la meta, diseñada específicamente para un problema de búsqueda particular.

$$h : \text{Estados} \rightarrow \mathbb{R}_{\geq 0}$$

En el contexto de búsqueda informada, $h(n)$ representa el **costo estimado** del camino más barato desde el estado n hasta la meta más cercana. Por convención, $h(\text{meta}) = 0$

En términos generales, una heurística es el estudio de métodos y reglas de descubrimiento e invención. En IA, nos interesa como *aproximación informada* que:

- usa conocimiento del dominio para guiar la búsqueda.
- descarta/remueve posibilidades de solución poco prometedoras
- no garantiza exactitud, pero sí eficiencia práctica
- **distancia en línea recta:** útil en mapas de carreteras. $h(n)$ = distancia euclidiana de n a la meta.
- **distancia manhattan:** útil en grillas. $h(n) = \sum_i |x_i - x_{\text{meta}}| + |y_i - y_{\text{meta}}|$
- **celdas fuera de lugar:** útil en el 8-puzzle. $h(n)$ = número de piezas no en su posición goal.

4.1. Repaso algoritmos de búsqueda informada

Los algoritmos estudiados en la clase anterior usan $h(n)$ de distintas formas:

Algoritmo	Función de evaluación	Estrategia
Greedy Best-First	$f(n) = h(n)$	Minimizar costo restante
A^*	$f(n) = g(n) + h(n)$	Minimizar costo total
A^* con peso	$f(n) = g(n) + w \cdot h(n)$	Sesgar hacia la meta

donde $g(n)$ es el costo acumulado desde el inicio hasta n (*backward*), y $h(n)$ es la estimación del costo restante (*forward*).

5. Admisibilidad, Monotonicidad, Informedness (Desigualdad triangular)

5.1. Admisibilidad

Definición 5.1 (Admisibilidad). Una heurística h es **admisible** (u *optimista*) si nunca sobreestima el costo real para llegar a la meta.

$$h(n) \leq h^*(n), \quad \forall n$$

donde $h^*(n)$ es el costo verdadero del camino óptimo desde n hasta la meta.

Consecuencia clave: si h es admisible, A^* sobre árbol de búsqueda es *optimo*: nunca descarta la solución óptima porque nunca penaliza en exceso ningún nodo.

Ejemplo 5.1 (Admisibilidad en BFS). BFS usa $f(n) = g(n)$ (equivalente a $h(n) = 0$ para todo n). Como $0 \leq h^*(n)$ siempre, $h \equiv 0$ es admisible (aunque trivial e ineficiente).